

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## ASSESSMENT TOOL 1 – Coding Challenge Task

|                  |                                                                              |               |                                           |
|------------------|------------------------------------------------------------------------------|---------------|-------------------------------------------|
| Course Code:     | CSA06                                                                        | Course Title: | Design and Analysis of Algorithms         |
| CO Assessed:     | CO4: Articulation (BL4, BL5)                                                 | Assessment:   | Assessment Tool 1 – Coding Challenge Task |
| Weightage:       | 40% of CIA                                                                   | Total Marks:  | 100                                       |
| CO4 Statement:   | Solve optimization problems using dynamic programming and greedy strategies. |               |                                           |
| Student Name:    | K. HEMASREE                                                                  | Reg. No.:     | 19241156                                  |
| Section / Batch: | SLOT C                                                                       | Date:         | 21/08/26                                  |

### Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

| Question Type         | Focus Area                                                                                      | Questions |
|-----------------------|-------------------------------------------------------------------------------------------------|-----------|
| Coding Challenge Task | Focus on Code and analyze optimization problems using dynamic programming and greedy strategies | Q1        |

### SECTION A – Coding Challenge Task

#### Code of Conduct

I K. HEMAREE(192411156) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

### RUBRICS FOR CALCULATION

#### Coding Challenge Task

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|----------|-----------|---------------------|----------------------|----------------------|---------------------|
|----------|-----------|---------------------|----------------------|----------------------|---------------------|

|                         |     |                                                                                           |                                                           |                                               |                                                  |
|-------------------------|-----|-------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------|--------------------------------------------------|
| Problem Understanding   | 20% | Clearly understands problem constraints, edge cases, and competitive requirements         | Understands problem with minor gaps in constraints        | Partial understanding; misses key constraints | Misinterprets problem or constraints             |
| Algorithm               | 25% | Selects optimal algorithm with best time and space complexity for competitive constraints | Uses efficient algorithm with minor scope for improvement | Uses basic/inefficient approach               | No clear algorithmic approach                    |
| Code Implementation     | 20% | Writes correct, efficient, and clean code that passes all constraints                     | Code works with minor errors or inefficiencies            | Partially correct code; fails for some cases  | Code does not execute or gives incorrect results |
| Competitive Performance | 20% | Solves problem within time limits and handles large inputs effectively                    | Solves within acceptable time with minor delays           | Struggles with time limits or larger inputs   | Unable to solve within time constraints          |
| Test Case Handling      | 15% | Handles all test cases including edge and hidden cases                                    | Handles most test cases                                   | Misses important edge cases                   | Fails on multiple test cases                     |

### Marks Evaluation:

| Criteria                | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|-------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Problem Understanding   | 20% |                     |                      |                      |                     |
| Algorithm               | 25% |                     |                      |                      |                     |
| Code Implementation     | 20% |                     |                      |                      |                     |
| Competitive Performance | 20% |                     |                      |                      |                     |
| Test Case Handling      | 15% |                     |                      |                      |                     |
| Total Marks (100):      |     |                     |                      |                      |                     |

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
| Name:             | Name:              | Name:              |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |

### Q1. Optimal Binary Search Tree (OBST)

#### Aim:

Minimize the **expected search cost** of a BST using **Dynamic Programming** based on given key access frequencies.

#### Algorithm (DP):

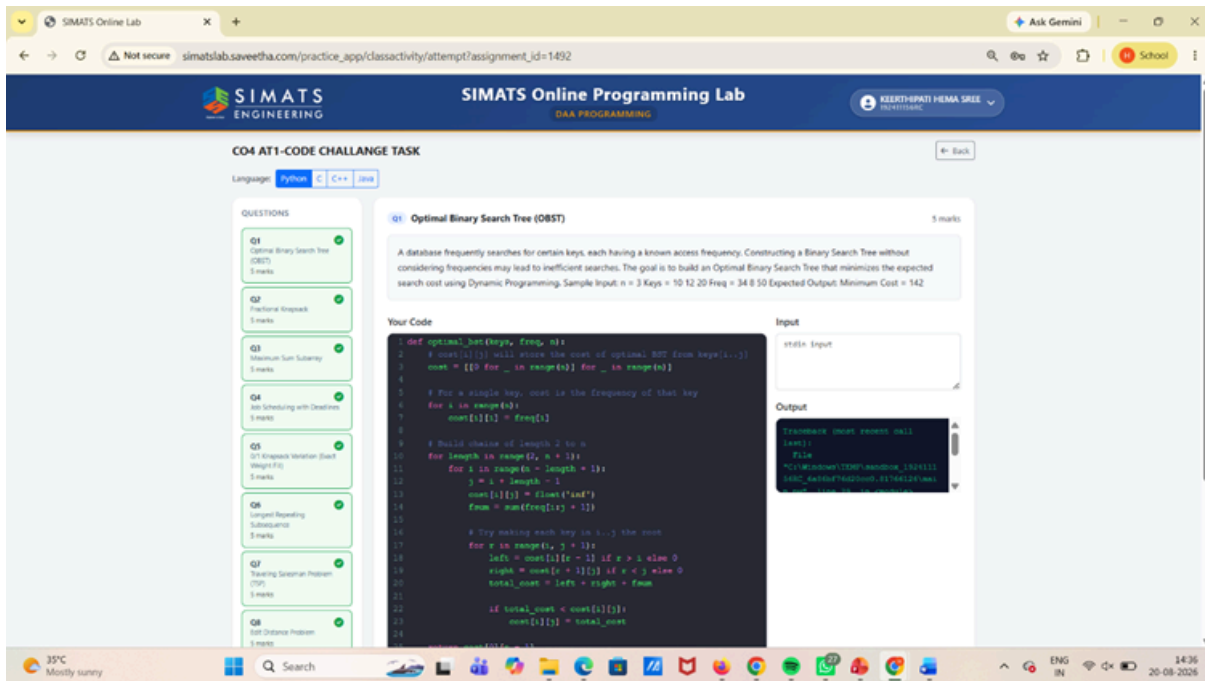
- Let  $\text{cost}[i][j]$  = minimum cost for keys  $i..j$ .
- For single key:  $\text{cost}[i][i] = \text{freq}[i]$ .
- For increasing chain lengths (2 to n):
  - For each interval  $[i..j]$ :
    - Compute  $\text{fsum} = \text{sum}(\text{freq}[i..j])$ .
    - Try every root  $r$  from  $i..j$ :
      - $\text{left} = \text{cost}[i][r-1]$ ,  $\text{right} = \text{cost}[r+1][j]$  (0 if out of range)
      - $\text{cost}[i][j] = \min(\text{cost}[i][j], \text{left} + \text{right} + \text{fsum})$
- Answer =  $\text{cost}[0][n-1]$ .

#### Output:

Print minimum expected cost.

Example (from prompt):

**Minimum Cost = 142**



## Q2. Fractional Knapsack

### Aim:

Maximize total profit when you can take fractions of items.

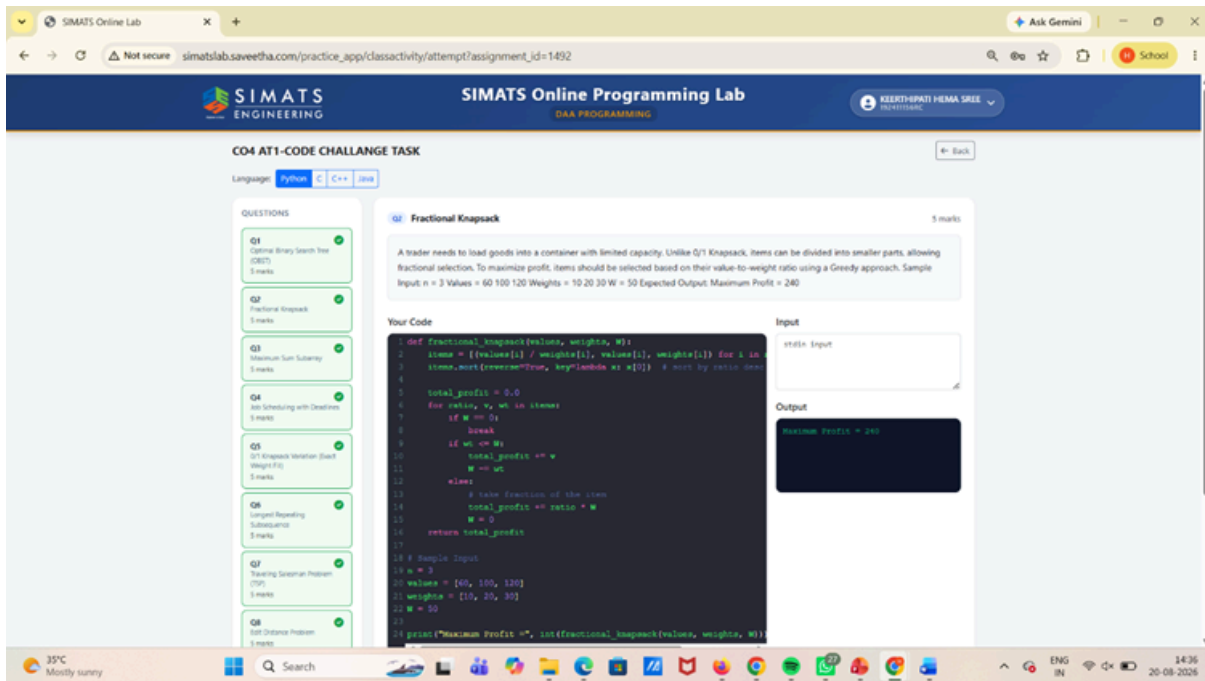
### Algorithm:

1. Compute profit/weight ratio for each item.
2. Sort items in decreasing order of ratio.
3. Take items greedily:
  - If full item fits, take it fully.
  - Else take remaining capacity fraction.
4. Sum profit.

### Output:

Maximum profit value.

(Example output depends on given input.)



### Q3. Maximum Sum Subarray (Kadane's)

#### Aim:

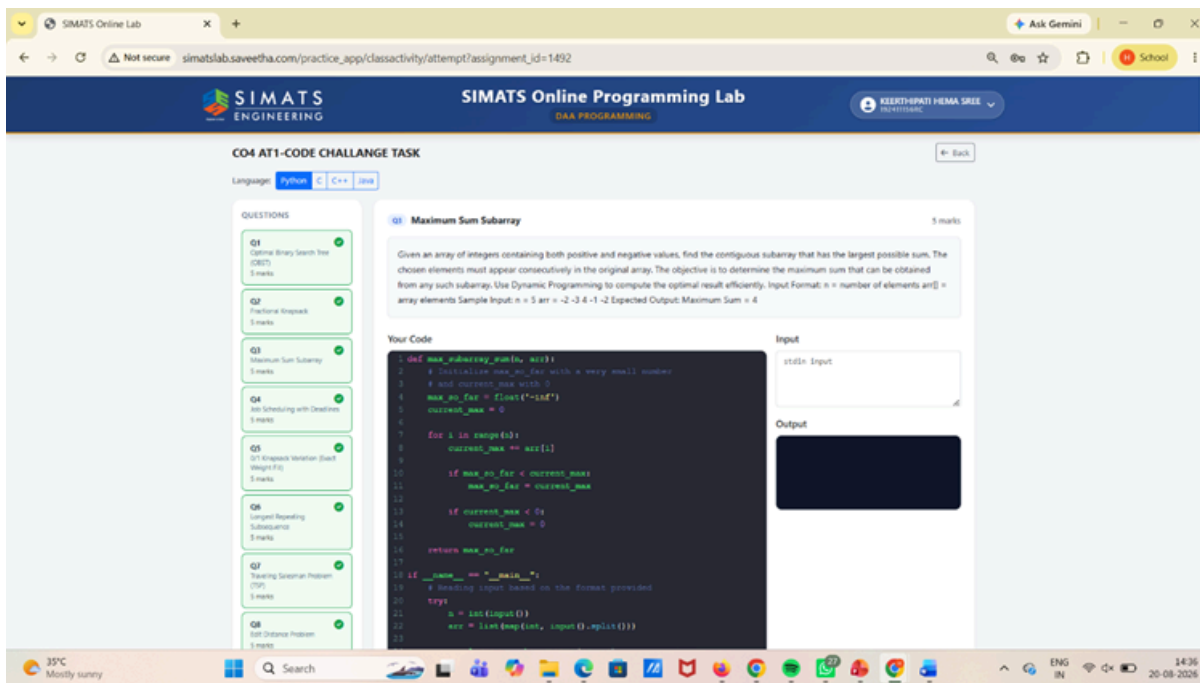
Find the subarray with maximum possible sum.

#### Algorithm:

1. Initialize max\_so\_far = arr[0], curr = arr[0].
2. For each element:
  - curr = max(arr[i], curr + arr[i])
  - max\_so\_far = max(max\_so\_far, curr)
3. Output max\_so\_far.

#### Output:

Print maximum subarray sum.



#### Q4. Job Scheduling with Deadlines

##### Aim:

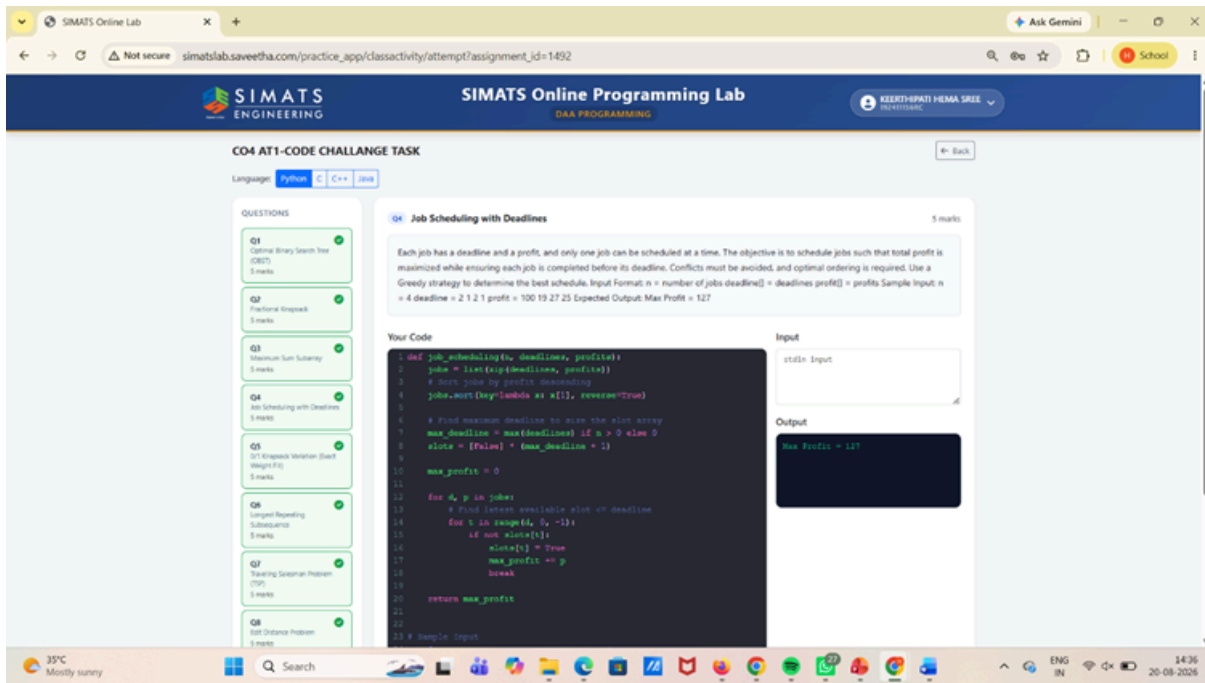
Schedule jobs before deadlines to maximize total profit.

##### Algorithm:

1. Sort jobs by decreasing profit.
2. Create an array slots of size = maximum deadline.
3. For each job:
  - Place it in the latest free slot before its deadline.
4. Count selected jobs and total profit.

##### Output:

Total maximum profit (and scheduled jobs if asked).



## Q5. 0/1 Knapsack Variation (Exact Weight Fill)

**Aim:**

Fill knapsack to **exact weight W** using **0/1 items** (each item once), maximizing value or checking feasibility.

**Algorithm:**

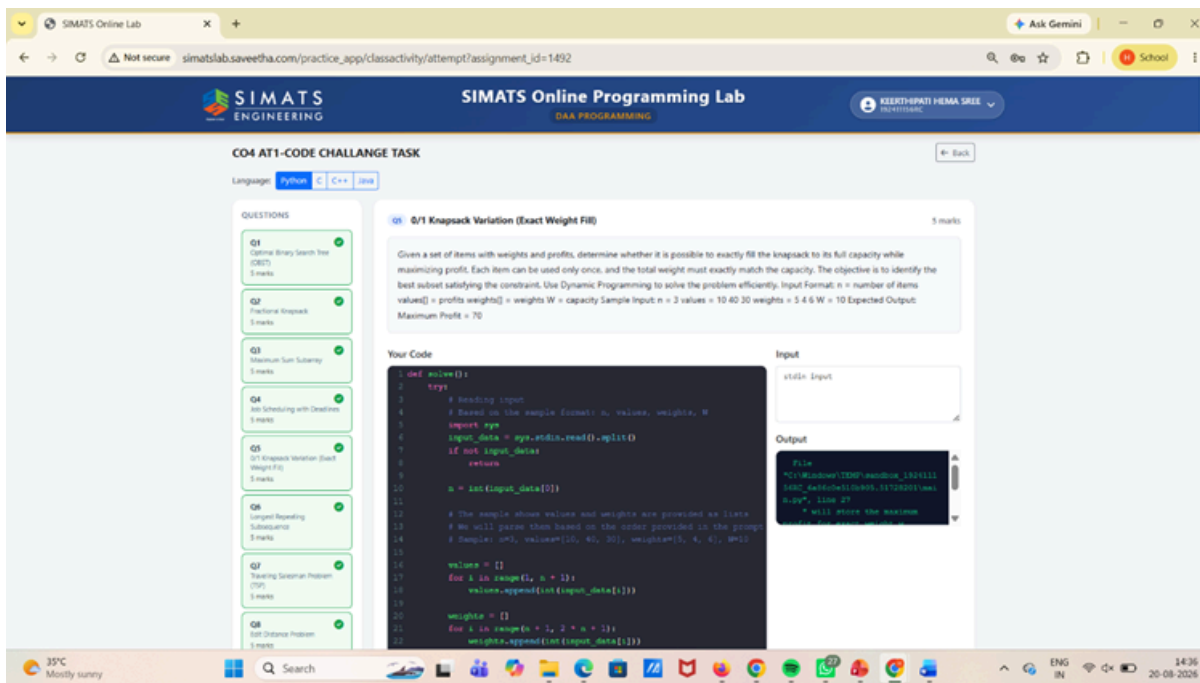
1. DP array `dp`

= best value for exact weight  $w$ .

2. Initialize  $dp[0]=0$ , others as  $-\infty$  (unreachable).
3. For each item (weight, value):
  - iterate  $w$  from  $W$  down to weight:
    - $dp = \max(dp, dp[w - \text{weight}] + \text{value})$
4. Answer =  $dp[W]$  (if unreachable, print no solution).

**Output:**

Best value for exact weight  $W$  (or “Not possible”).



## 1. Q6. Longest Repeating Subsequence

### Aim:

Find the **longest repeating subsequence** in a string.

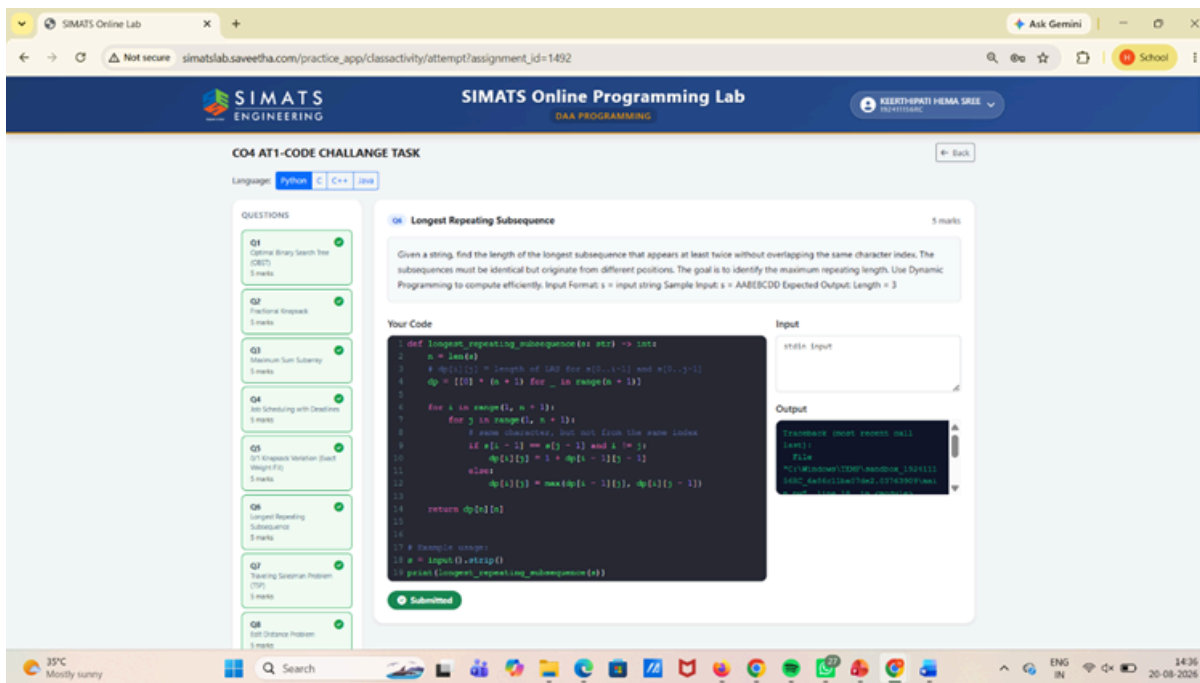
### Algorithm:

2. Let  $s$  be the string of length  $n$ .
3. Use DP table  $dp[i][j]$ .
4. If  $s[i-1] == s[j-1]$  and  $i != j$ :
  - $dp[i][j] = 1 + dp[i-1][j-1]$
5. Else:
  - $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$
6. Answer =  $dp[n][n]$ .

### Output:

Length of longest repeating subsequence.





## 1. Q7. Traveling Salesman Problem (TSP)

### Aim:

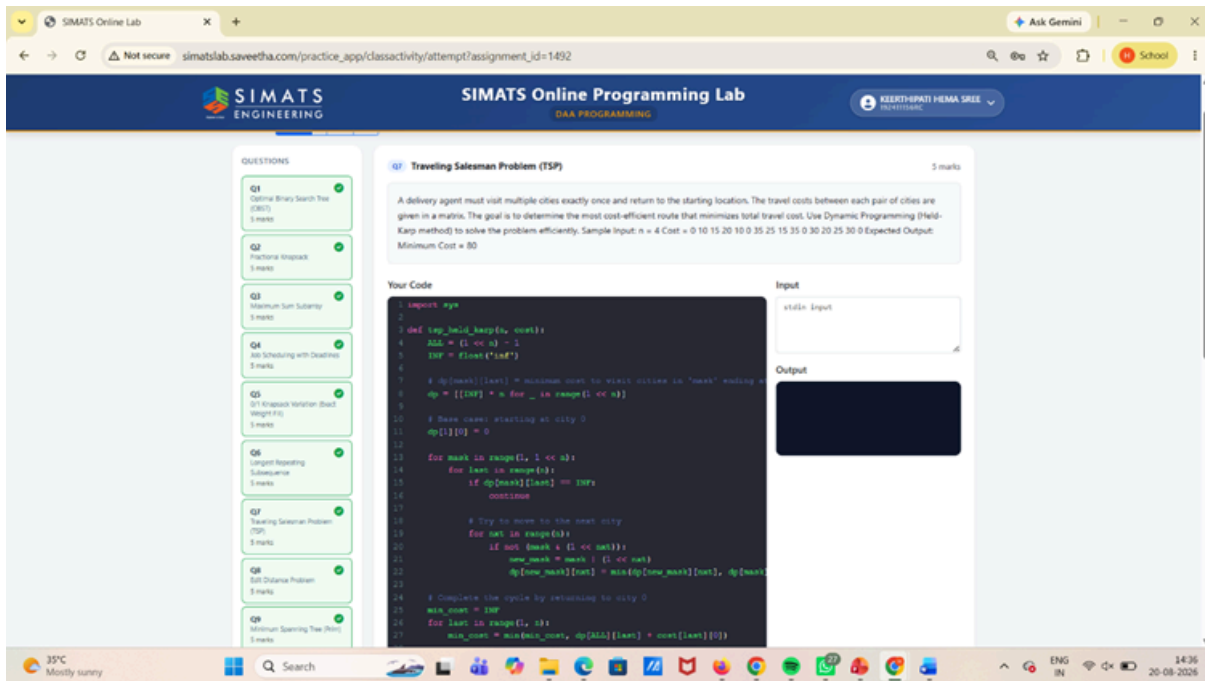
Find the minimum cost Hamiltonian cycle (visit every city exactly once and return).

### Algorithm (Typical DP for TSP - Bitmask):

2. Use DP:  $dp[mask][i] = \text{min cost to reach city } i \text{ having visited set } mask$ .
3. Base:  $dp[1 \ll start][start] = 0$ .
4. Transition:
  - For each mask, for each  $i$  in mask:
    - try next  $j$  not in mask:
      - $dp[mask | (1 \ll j)][j] = \min(dp[...], dp[mask][i] + cost[i][j])$
5. Final: add return edge back to start.

### Output:

Minimum tour cost.



## 1. Q8. Edit Distance Problem

### Aim:

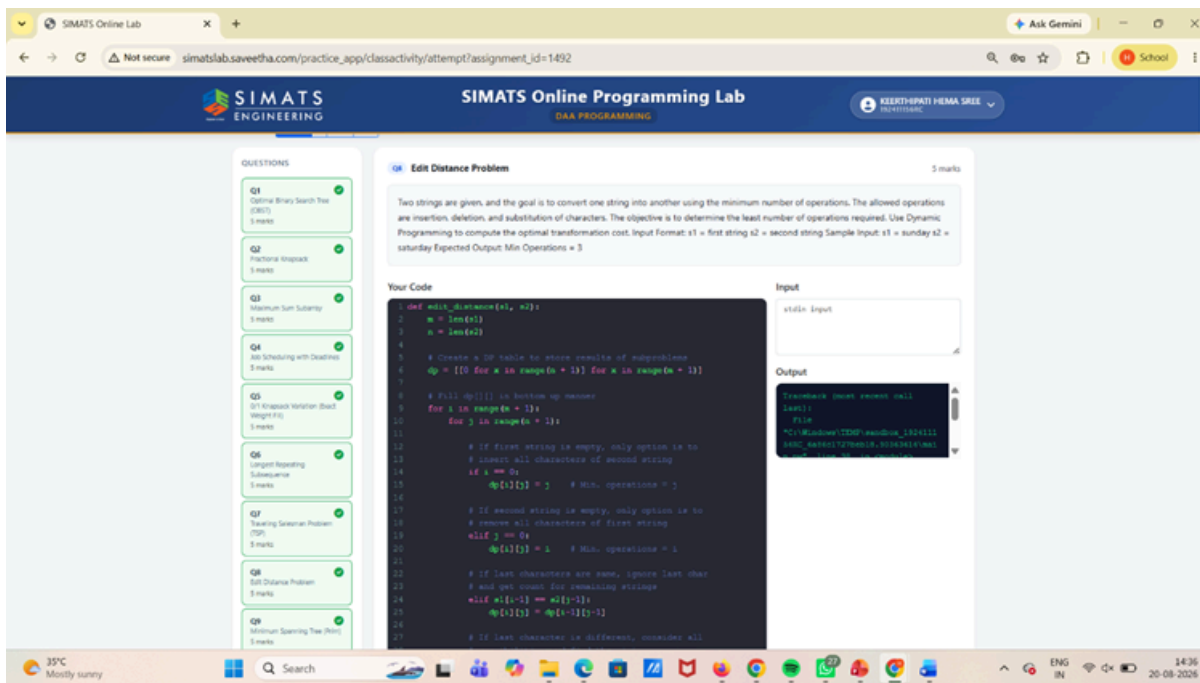
Compute minimum operations to convert string A to B:  
(insert, delete, replace)

### Algorithm:

2. Let  $dp[i][j]$  = min edits from  $A[0..i-1]$  to  $B[0..j-1]$ .
3. Base:
  - $dp[i][0] = i$  (delete all)
  - $dp[0][j] = j$  (insert all)
4. If  $A[i-1] == B[j-1]$ :
  - $dp[i][j] = dp[i-1][j-1]$
5. Else:
  - $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$
6. Answer =  $dp[m][n]$ .

### Output:

Minimum number of edits.



## Q9. Minimum Spanning Tree (Prim's)

### Aim:

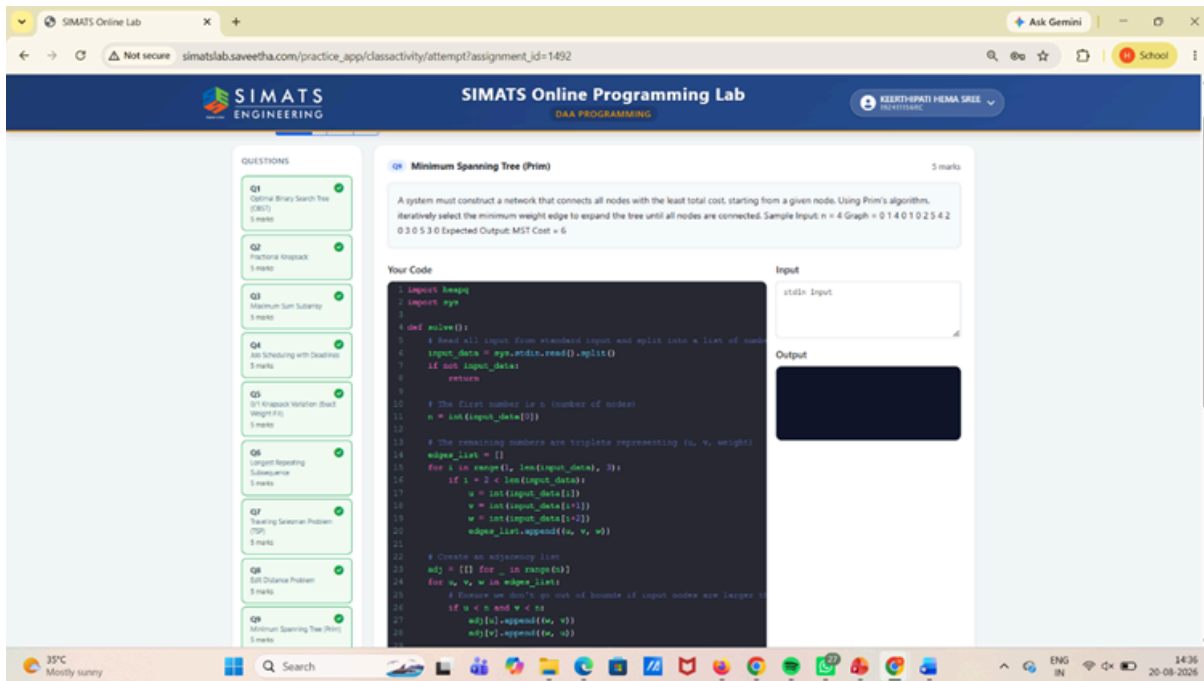
Find MST with minimum total weight for a connected weighted graph.

### Algorithm:

7. Start from any node.
8. Maintain:
  - $key[v]$  = min edge weight to connect  $v$  to MST
  - $parent[v]$  = MST parent
9. Repeat:
  - choose vertex  $u$  not in MST with smallest  $key[u]$
  - add  $u$  to MST
  - update keys of adjacent vertices
10. MST edges are from  $parent[]$ .

### Output:

Total MST weight and edges.



## 1. Q10. Minimum Cost to Merge Stones

### Aim:

Merge stones into one pile with minimum total cost.

Cost of merging a range = sum of stones in that range.

### Algorithm (DP with Prefix Sum):

2. Precompute prefix sums  $prefix[]$  so range sum is fast.
3.  $dp[i][j]$  = minimum cost to merge stones from  $i$  to  $j$ .
4. Length from 2 to  $n$ :
  - $dp[i][j] = \min(dp[i][k] + dp[k+1][j] + \text{sum}(i..j))$
5. Answer =  $dp[0][n-1]$ .

### Output:

Minimum cost to merge all stones.

